

< 6 월 23 일 보고서 >

연구를 본격적으로 시작하기 앞서, 연구 주제인 시스톨릭 어레이 기반 행렬곱 가속기 설계 및 비교 분석에서 시스톨릭 어레이와 AI 에 관한 이론적 배경을 쌓는 과정을 진행함.

1. Systolic Array 복습

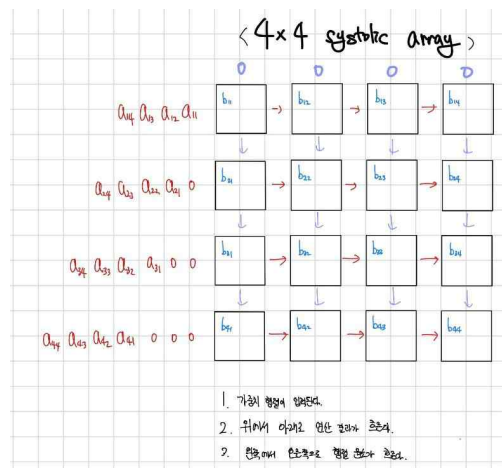
Systolic Array 는 데이터가 정형화된 격자 형태의 PE 배열을 통해 주기적으로 흐르도록 설계된 하드웨어 아키텍처이다.

- 데이터 입력 및 고정 방식에 따른 분류

Systolic Array 는 행렬 연산 과정에서 가중치, 입력 데이터, 또는 연산 결과를 PE 내부에 고정하느냐, 아니면 흐르게 하느냐에 따라 크게 두 가지 구조로 나뉘게 된다. 그 중에서 연구 계획에 따른 weight stationary 를 복습하였다.

① Weight Stationary

- 개념: 행렬 곱셈의 한 축을 담당하는 가중치 데이터를 PE 내부 레지스터에 미리 저장해 두고, 입력 데이터와 누적 제어 신호를 어레이의 외부에서 내보내며 흐르게 하는 방식이다.
- 입력 데이터가 왼쪽에서 오른쪽, 연산 결과는 위에서 아래로 이동한다.



2. 머신러닝 및 딥러닝 학습

Coursera 사이트에서 기초적인 머신러닝에 대한 개념과 원리 그리고 수식적으로 접근하는 이해를 하였다. Supervised learning 과 Unsupervised learning 의 학습과정에서 현재까지는 supervised learning 의 회귀와 분류를 unsupervised learning 의 클러스터링 알고리즘까지 개념을 쌓고, 선형 회귀 모델의 수식적인 표현까지 이해하였다.

< 6 월 24 일 보고서 >

1. 진행 내용

6 월 24 일 회의에서는 행렬곱 가속기 구조를 이해하기 위해 먼저 행렬곱 계산 방식과 MAC 구조를 중심으로 학습하였다. 행렬곱은 각 출력 원소를 계산할 때 곱셈과 덧셈이 반복되는 구조이다. 따라서 하드웨어에서는 이를 단순한 곱셈 연산이 아니라, 곱셈 결과를 기존 누적합에 더하는 MAC 연산으로 표현할 수 있다.

MAC 은 Multiply-Accumulate 의 약자로, 기본 연산은 다음과 같다.

$$\text{sum} = \text{sum} + a \times b$$

예를 들어 4×4 행렬곱에서 출력 행렬 C 의 한 원소는 A 의 한 행과 B 의 한 열을 각각 곱한 뒤 모두 더해서 계산된다. 즉 하나의 출력 원소를 만들기 위해 여러 번의 곱셈과 누적 덧셈이 필요하다. 이에 따라 각자 MAC 구조, systolic array 구조, array multiplier 구조를 나누어 학습하고, 4×4 행렬곱을 하드웨어로 구현하기 위한 코드 초안을 작성하였다.

2. 각자의 행렬곱 및 곱셈기 구현 방식

(1) 이예준: 파이프라인형 MAC 구조

이예준의 구현은 4×4 행렬곱에서 하나의 출력 원소가 네 번의 곱셈과 누적 덧셈으로 계산된다는 점에 집중하였다. 이를 위해 psum0, psum1, psum2 와 같은 중간합 변수를 사용하였고, 시간 변수 t 에 따라 각 행의 결과가 순차적으로 계산되도록 구성하였다.

이 방식은 각 단계에서 곱셈 결과를 중간합에 저장하고, 다음 단계에서 새로운 곱셈 결과를 더하는 방식으로 동작한다. 따라서 행렬곱이 MAC 연산의 반복으로 이루어진다는 점을 이해하기 쉽고, 2×2 에서 학습한 MAC 구조를 4×4 로 확장하는 과정에 적합하였다.

다만 이 구조는 실제 PE 들이 격자 형태로 연결된 systolic array 라기보다는, MAC 연산이 시간에 따라 누적되는 과정을 파이프라인처럼 표현한 구조에 가깝다. 따라서 PE 간 데이터 이동이나 weight-stationary dataflow 를 명확히 보여주지는 못한다는 한계가 있다. 이 부분은 주형능의 systolic array 구조를 통해 보완할 수 있었다.

(2) 주형능: 4×4 Systolic Array 구조

주형능의 구현은 4×4 PE 를 격자 형태로 배치하는 systolic array 구조를 나타내는 방식이다. 이 구조에서 각 PE 는 왼쪽에서 activation 값을 입력받아 오른쪽으로 전달하고, 위쪽에서 partial sum 을 입력받아 아래쪽으로 전달한다. 또한 weight 값은 각 PE 에 고정되어 있으며, PE 내부에서 MAC 연산을 수행하는 구조를 의도하였다.

이 방식은 systolic array 의 기본적인 데이터 흐름을 이해하기에 적합하였다. 특히 a_wire 와 p_wire 를 사용하여 activation 과 partial sum 이 서로 다른 방향으로 이동하는 구조를 표현하였다. 이를 통해 단순한 MAC 계산이 아니라 여러 개의 PE 가 연결되어 병렬적으로 계산하는 구조를 이해할 수 있었다.

다만 PE 내부에서 실제 곱셈과 누적 덧셈을 수행하는 구체적인 구현이 별도로 필요하고, 입력 activation 을 시간차를 두고 주입하는 skewing 구조가 완전히 포함되어 있지는 않다는 한계가 있었다. 이 부분은 배운재의 PE 구조와 top module 구성을 참고하여 보완할 수 있었다.

(3) 배운재: Array Multiplier 기반 Weight-Stationary Systolic Array

배운재의 구현은 직접 설계한 array_multiplier_signed 를 PE 내부에 넣고, 이를 4×4 systolic array 로 확장하는 방식이다. 각 PE 는 weight 값을 내부에 저장하고, 입력 activation 과 weight 를 곱한 뒤 위쪽에서 전달된 partial sum 에 더하여 아래쪽으로 전달한다. 이러한 구조는 weight 를 PE 내부에 고정시키고 activation 과 partial sum 을 이동시키는 weight-stationary 방식에 해당한다.

또한 use_dsp = "no" 속성을 사용하여 FPGA 내부의 DSP 블록을 사용하지 않고 LUT 기반 곱셈기로 합성되도록 하였다. 이는 직접 구현한 곱셈기 구조의 특성을 Vivado 에서 비교하기 위한 목적이다.

이 방식은 곱셈기, PE, systolic array, top FSM 구조까지 포함하고 있어 실제 Vivado 실험에서 LUT, FF, DSP, WNS, Fmax, Power 등을 비교하기에 가장 적합하다. 다만 코드 구조가 상대적으로 복잡하고, activation 이 올바른 cycle 에 주입되는지, 결과가 정확한 cycle 에 저장되는지는 testbench 를 통해 검증해야 한다는 한계가 있다. 이 부분은 이예준의 단순 MAC 구조와 결과를 비교하면서 계산 정확성을 검증하는 방향으로 보완할 수 있었다.

(4) 이재현: Signed Array Multiplier 단위 구현

이재현의 구현은 8 비트 signed 입력 두 개를 받아 16 비트 signed 곱셈 결과를 출력하는 array multiplier 구조이다. multiplier 의 각 비트를 확인하여 해당 비트가 1 인 경우 multiplicand 를 왼쪽으로 shift 한 값을 누적합에 더하는 shift-add 방식으로 동작한다.

signed 2 의 보수 표현에서는 최상위 비트가 음수 가중치를 가지므로, 마지막 부호 비트에 해당하는 항은 더하지 않고 빼는 방식으로 처리하였다. 또한 use_dsp = "no" 속성을 사용하여 FPGA 내장 DSP 블록이 아닌 LUT 기반 곱셈기로 합성되도록 하였다.

이 구조는 Wallace tree multiplier 는 아니지만, array multiplier 기반 비교 실험의 기준 곱셈기로 사용할 수 있다. 한계는 부분곱을 병렬적으로 압축하는 Wallace tree 구조가 포함되어 있지 않다는 점이다. 따라서 이후 Wallace tree 와 비교하기 위해서는 full adder 와 half adder 를 이용한 partial product compression 구조를 별도로 구현할 필요가 있다.

3. 서로의 한계와 보완 방식

각자의 구현은 서로 다른 관점에서 작성되었기 때문에, 단독으로는 한계가 있었지만 서로의 부족한 부분을 보완하는 방향으로 발전할 수 있었다.

이예준의 코드는 MAC 연산의 흐름을 이해하기 쉽다는 장점이 있다. 하나의 출력 원소가 여러 번의 곱셈과 누적 덧셈으로 계산되는 과정을 psum 변수를 통해 단계적으로 보여주었다. 하지만 실제 systolic array 처럼 PE 들이 격자로 연결되어 데이터를 주고받는 구조는 아니었다. 따라서 PE 간 데이터 이동 방식과 systolic array 의 구조적 특징은 주형능의 코드로 보완할 수 있었다.

주형능의 코드는 4×4 PE array 구조를 통해 activation 이 왼쪽에서 오른쪽으로 이동하고 partial sum 이 위에서 아래로 이동하는 systolic array 의 dataflow 를 잘 보여주었다. 그러나 단독 연산 형태로 외부 핀 수 초과 및 보드 단독 회로의 결합 부분이 부족하였다. 이 부분은 배운재의 pe_ws 와 matmul_top_ws 구조를 통해 보완할 수 있었다.

배운재의 코드는 곱셈기, PE, systolic array, top FSM 까지 포함하고 있어 실제 실험 구조로 사용하기 적합하였다. 그러나 코드가 복잡하고, 출력 타이밍과 activation 주입 타이밍을 검증해야 한다는 한계가 있었다. 이 부분은 이예준의 단순 MAC 구조와 결과를 비교하고, testbench 를 작성하여 계산 결과가 정확한지 확인하는 방식으로 보완할 수 있다.

이재현의 코드는 signed array multiplier 를 간단하고 명확하게 구현했다는 장점이 있다. 특히 DSP 를 사용하지 않고 LUT 기반 곱셈기를 구성할 수 있도록 하여, array multiplier 기반 비교 실험의 기준 구조로 활용할 수 있다. 다만 Wallace tree 구조는 아직 포함되어 있지 않으므로, 이후에는 이 구조를 기준으로 삼고 Wallace tree multiplier 를 추가로 구현하여 두 곱셈기의 자원 사용량과 성능을 비교하는 방향으로 나아가야 한다.

4. 다음에 할 일

다음 단계에서는 각자 작성한 코드 초안을 하나의 실험 흐름으로 정리해야 한다. 우선 4×4 int8 행렬곱에 대해 각 구조가 정확한 결과를 출력하는지 testbench 를 통해 검증해야 한다. 이후 Vivado 에서 합성 및 구현을 진행하여 자원 사용량과 성능을 비교한다.

구체적으로는 먼저 array multiplier 기반 weight-stationary systolic array 구조를 기준 구조로 정리한다. 그 다음 동일한 조건에서 Wallace tree multiplier 를 적용한 구조를 추가로 구현한다. 두 구조 모두 DSP 사용을 제한하여 LUT 기반 곱셈기로 합성되도록 하고, Vivado report 를 통해 LUT, FF, DSP, WNS, Fmax, Power 를 비교한다.

이후 결과를 표로 정리하여 곱셈기 구조 변화가 FPGA 자원 사용량과 타이밍 특성, 전력 소모에 어떤 영향을 주는지 분석한다. 특히 Power 비교가 명확하지 않을 경우에는 simulation activity 를 반영한 power analysis 를 추가로 진행할 필요가 있다.

최종적으로는 다음 항목을 중심으로 비교를 진행한다.

1. 4×4 int8 행렬곱 testbench 작성
2. 각 구조의 계산 결과 검증
3. array multiplier 기반 구조 합성 및 구현
4. Wallace tree multiplier 기반 구조 추가 구현
5. DSP 사용량이 0 인지 확인
6. LUT, FF, WNS, Fmax, Power 결과 비교
7. 결과 표 작성 및 구조별 장단점 분석

따라서 6 월 24 일 진행 내용은 행렬곱을 MAC 연산으로 이해하고, 이를 바탕으로 각자 다른 방식의 4×4 행렬곱 가속기 코드 초안을 작성한 단계로 정리할 수 있다. 이후에는 각자의 코드를 단순 학습용 구조와 실제 실험용 구조로 구분하고, 최종적으로 array multiplier 와 Wallace tree multiplier 를 동일한 조건에서 비교하는 방향으로 진행할 예정이다.

참고자료 V 파일로 첨부

< 6 월 25 일 보고서 >

1. 이전 상황

Array Multiplication 을 기반으로 만든 행렬곱 가속기를 전달 RTL 구현하여 Run Synthesis 를 통과하였다. 이때 우리가 구현한 모듈 중 matmul_top_ws 모듈에서는 코드를 Implementation 하기 위해 핀의 개수가 772 개가 필요하였다. 그러나 우리가 연구에 사용하는 FPGA 보드 속 핀의 개수는 210 개로 물리적으로 구현이 불가능한 상황이 발생하여 Run Implementation 을 수행할 때 에러가 발생하였다. 이때 우리는 FPGA 보드를 설계할 때 '코어'와 '래퍼(Wrapper)'를 분리하는 설계 방식을 적용하기로 결정하였고, 보드 래퍼를 설계하였다.

2. 보드 래퍼 설계

우리는 코어(matmul_top_ws)는 그대로 두고, 이를 감싸는 래퍼가 입력을 칩 안에 미리 저장해 두어 결과를 LED 와 7-Segment 에 보여주는 형식으로 설계하고자 하였다.

설계한 보드 래퍼, fpga_top_bram 은 모듈 matmul_top_ws 를 인스턴스 한다. 이 래퍼는 BRAM 에 저장된 입력 행렬 A 와 가중치 행렬 B 를 BRAM 에서 인출하여 코어에서 연산을 하고, 결과 행렬 C 를 7-segment 를 통해 보여준다. 이를 위해 행렬 A, B 를 .mem 파일로 BRAM 에 미리 저장해 두었고, 보드 조작을 위한 xdc 파일도 설계하였다.

다음은 우리가 보드 속 스위치와 LED 에 지정한 역할이다.

- BTNC : Start
- BTNU : Reset
- SW15 : Data Set (0 = test1, 1 = test2)
- SW3 ~ SW0 : 보고자 하는 원소의 인덱스 선택 (C 의 16 개 값 순서를 이진수로 표현)
- 7-Segment : 선택한 값을 부호가 있는 10 진수로 출력

3. 현재 상황

설계한 보드 래퍼로 인해 문제없이 Implementation 할 수 있었다. 그러나 fpga_top_bram 을 합성하였을 때 모듈 속에는 우리가 성능 비교를 위해 필요한 로직과는 무관한 무거운 부가 로직들이 섞여 있어 WNS 가 음수로 나오는 문제가 생겼다. 이로 인해 보드의 클럭인 100MHz 로는 실행할 수 없는 문제가 생겼다. 따라서 우리가 구현한 RTL 의 성능을 합성 단계 추정치로 확인하게 되는 상황에 이르렀다. 이러한 이유로 인해 우리가 구하고자 하는 전력 소비량은 합성 단계라 정확도가 낮고, 이를 해결하기 위해 기존의 보드 래퍼를 없애면 Implementation 을 못하게 되는 딜레마가 발생하였다. 따라서 우리는 Fmax 와 Power 를 모두

정량적으로 비교하기 위해 새로운 대안을 모색해야 했다. 따라서 측정 전용 래퍼인 `power_bench_wrapper` 를 설계하였다.

4. Power Bench Wrapper 설계

이 래퍼는 기존의 코어를 수정하지 않고 그대로 인스턴스 하여 측정에 필요한 세 가지 기능만을 추가한다. 먼저 이 래퍼는 입출력 핀을 축소한다. 코어의 입출력 핀을 외부로 노출하지 않고 외부 핀을 클럭, 리셋, 출력 LED 만으로 제한하였다. 이로 인해 기존의 문제였던 초과된 핀의 개수를 획기적으로 감소하여 Implementation 이 정상적으로 완료되는 것을 확인할 수 있었다. 또한 그 결과로부터 정확한 Power 와 Fmax 값을 얻었다. 이를 기반으로 추후에 구현할 Wallace Tree 곱셈 구조와 정량적으로 비교할 수 있을 것이라 예상하였다.

다음으로 래퍼는 LFSR 기반 입력을 생성한다. 입력 행렬을 고정된 상수로 두었을 때 두 가지 문제가 발생하였다. 첫째로 합성 도구가 상수 입력에 대해 상수 전파 최적화를 수행하여 곱셈기 회로 자체를 상수로 접어버리므로 측정 대상인 곱셈기가 합성 결과에서 소멸해버린다. 둘째로는 입력이 변하지 않으면 회로의 toggle 활동이 없어 Dynamic Power 가 비현실적으로 낮게 측정된다. 따라서 이를 방지하기 위해 LFSR(Linear Feedback Shift Register)을 이용해 매 연산마다 입력 값을 난수로 변화시켰다. 이로 인해 곱셈기 회로가 보존되고, 지속적인 신호 토글이 발생하여 동적 전력이 정상적으로 측정된다.

마지막으로 래퍼는 연산 자동 반복을 수행한다. 코어는 한 번의 행렬곱을 수행한 뒤 정지하므로, 이 상태에서는 이 상태에서는 지속적인 신호 토글이 발생하지 않는다. 따라서 래퍼는 연산 완료를 감지하면 코어를 자동으로 재시작 시켜 행렬곱을 연속 반복하게 한다. 이를 통해 측정 구간 동안 토글이 지속되어 전력 측정의 안정성을 확보할 것이라 생각하였다.

따라서 보드를 시현할 때는 `fpga_top_bram` 을 top 으로 설정하였고, Power 와 Fmax 를 측정할 때는 `power_bench_wrapper` 를 top 으로 설정하였다.

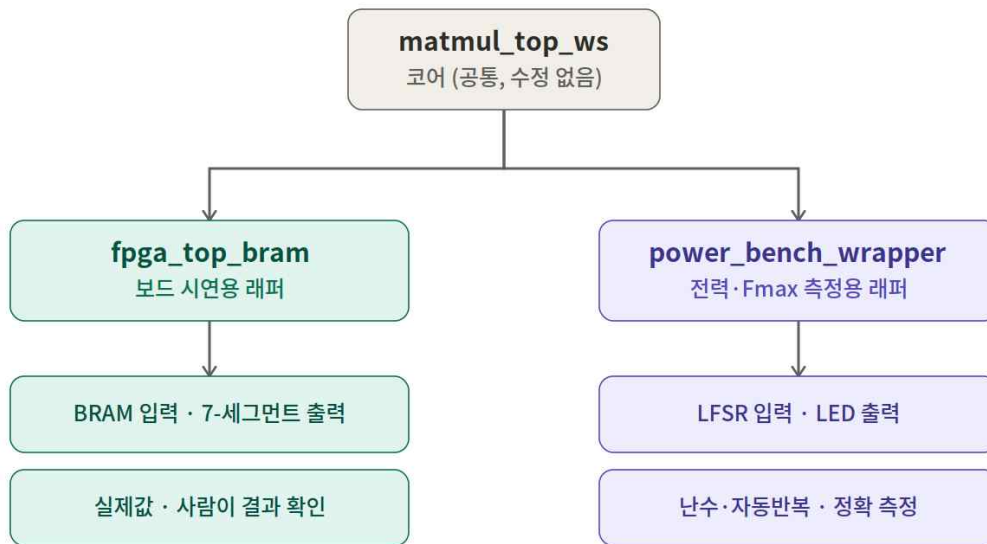


그림 1

5. 이후 계획

이후 Wallace Tree 곱셈기를 사용하는 행렬곱 가속기를 구현하고 LUT, FF, Power, Fmax 를 측정하여 이전에 설계한 Array 곱셈기를 기반 행렬곱 가속기와 비교하기로 계획하였다.

< 6 월 26 일 보고서 >

1. Wallace Tree 구조 구현

```
module csa #(
    parameter W = 16
) (
    input wire [W-1:0] a,
    input wire [W-1:0] b,
    input wire [W-1:0] c,
    output wire [W-1:0] sum,
    output wire [W-1:0] carry
);
    assign sum = a ^ b ^ c;
    assign carry = (a & b) | (b & c) | (c & a);
endmodule
```

1) Full adder

각 자릿수에서의 덧셈을 위한 Full Adder

출력값 : 1. Sum, 2. carry

// 2. 부분 곱 배열 생성

```
wire signed [2*W-1:0] pp [0:W-1];
```

```
genvar i;
```

```
generate
```

```
    for (i = 0; i < W-1; i = i + 1) begin : pp_gen
```

```
        assign pp[i] = b[i] ? (ext_a <<< i) : 0;
```

```
    end
```

```
    assign pp[W-1] = b[W-1] ? -(ext_a <<< (W-1)) : 0;
```

```
endgenerate
```

2) 부분 곱 배열 생성

LSB ~ (MSB-1) : '1'일 경우 곱해서 각 pp 에 저장할 때 더한다.

MSB : 2's 보수 체계에 따라 '1'일 경우 pp 에 저장할 때 뺀다.

```

// [1단계] 8->6
wire [2*W-1:0] s1_1, c1_1, s1_2, c1_2;
csa #(.W(2*W)) csa1_1 (pp[0], pp[1], pp[2], s1_1, c1_1);
csa #(.W(2*W)) csa1_2 (pp[3], pp[4], pp[5], s1_2, c1_2);

// [2단계] 6->4
wire [2*W-1:0] s2_1, c2_1, s2_2, c2_2;
csa #(.W(2*W)) csa2_1 (s1_1, (c1_1 << 1), s1_2, s2_1, c2_1);
csa #(.W(2*W)) csa2_2 ((c1_2 << 1), pp[6], pp[7], s2_2, c2_2);

// [3단계] 4->3
wire [2*W-1:0] s3_1, c3_1;
csa #(.W(2*W)) csa3_1 (s2_1, (c2_1 << 1), s2_2, s3_1, c3_1);

// [4단계] 3->2
wire [2*W-1:0] s4_1, c4_1;
csa #(.W(2*W)) csa4_1 (s3_1, (c3_1 << 1), (c2_2 << 1), s4_1, c4_1);

assign p = s4_1 + (c4_1 << 1);

```

3) Wallace Tree Reduction & 최종 결과값 도출

1 단계 : 8 개의 줄을 3 줄씩 2 그룹으로 묶어 6 개로 줄임 (사용되지 않은 pp[6], pp[7]은 그대로 다음단계로 넘어간다.)

2 단계 : 6 개의 줄을 3 줄씩 2 그룹으로 묶어 4 개로 줄임

3 단계 : 4 줄 중 3 줄만 묶어 csa 에 넣는다.

4 단계 : 마지막 남은 3 줄을 묶어 csa 에 넣은 후 최종 출력을 2 줄로 줄인다.

최종 덧셈 : 위 압축 단계를 거친 최종 2 줄은 더 이상 압축 불가하기에 일반 덧셈기호(+)로 덧셈을 수행한다.

2. Array multiplier VS Wallace Tree 성능 비교

동일한 testbench 로 성능을 비교함 (기준 clk : 100MHz)

A = [{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,15,16}]

B = Identity Matrix

	Array	Wallace Tree
LUT	1734	1647
FF	1106	1106
DSP	0	0
WNS[ns]	2.428	2.143
Fmax[MHz]	132.06	127.27
Power[W]	0.093	0.093